UNIVERSIDADE FUMEC
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DESENVOLVIMENTO WEB FRONT-END
PROF. DÂNGELO SILVA MIRANDA

KAIQUE FERREIRA PEIXOTO

CARDVAULT JP — TRABALHO 02

Aplicação Angular: componentes, rotas, serviços e data-binding

Belo Horizonte
2026

1. INTEGRANTES

Kaique Ferreira Peixoto (trabalho individual).

2. DESCRIÇÃO DA APLICAÇÃO

Nome: CardVault JP.

Objetivo: oferecer a colecionadores e lojistas um ambiente único para gestão, catalogação, avaliação e negociação de cartas colecionáveis de Pokémon TCG, com foco em coleções japonesas e itens de nicho. Nesta segunda entrega, o protótipo HTML/CSS do Trabalho 01 foi convertido em uma aplicação Angular 22 baseada em `NgModule`, com seis páginas roteadas, um modelo de dados tipado, dados simulados (mock) e serviços que concentram toda a regra de negócio. As cartas, nomes e coleções são fictícios e as artes são gradientes CSS, sem imagens ou nomes oficiais.

Principais funcionalidades desta versão:

- Catálogo com busca por nome, número ou coleção e filtros combináveis de raridade, tipo e coleção;
- Página de detalhe acessada pela rota parametrizada `/cards/:id`, com atributos, valor em reais, estado de conservação, histórico de preço em SVG e ações de negociar, editar e voltar;
- Área administrativa com CRUD completo (cadastrar, editar e excluir com confirmação) e formulário com validação;
- Dashboard com indicadores calculados pelo serviço (total, unidades, valor do acervo, valor médio), gráfico por coleção, distribuição por raridade e top 5 cartas mais valiosas;
- Login com perfis Cliente e Administrador (usuários mock), cadastro de novo cliente e página institucional com formulário de contato validado;
- Navbar com `routerLink` / `routerLinkActive` e identidade visual do protótipo (fundo #0F1220, superfícies #1A1F35, destaque #E3350D, dourado #F2C14E, texto #F5F5F7, fonte Inter, botões com 8 px de raio).

3. ARQUITETURA DESTA VERSÃO

3.1 Estrutura de pastas

```
cardvault-jp/
├── prototipo/           (Trabalho 01)
├── docs/                prints/ + este relatório
└── app/                 projeto Angular 22 (NgModule)
    ├── angular.json, package.json, tsconfig*.json
    └── src/
        ├── index.html, main.ts, styles.css   (identidade visual global)
        └── app/
            ├── app.module.ts                 declarations, FormsModule, pt-BR
            ├── app-routing.module.ts         rotas da aplicação
            ├── app.component.*               <app-navbar> + <router-outlet>
            ├── models/    card.model.ts, usuario.model.ts
            ├── mocks/     cards.mock.ts
            ├── services/  card.service.ts, auth.service.ts
            ├── pipes/     conservacao.pipe.ts
            ├── components/navbar/
            └── pages/     catalogo/, detalhe-card/, admin-cards/,
                           dashboard/, sobre/, login/
```

O projeto foi criado com `ng new cardvault-jp --no-standalone --routing --style=css --ssr=false`. Todos os componentes são declarados no `AppModule` (`standalone: false`). Os arquivos gerados pelo CLI 22 com o novo padrão de nomes (`app-module.ts`, `app-routing-module.ts`) foram renomeados para a convenção `app.module.ts` e `app-routing.module.ts`.

3.2 Model `Card` (src/app/models/card.model.ts)

Campo	Tipo	Descrição
id	number	Código interno (gerado pelo serviço no cadastro)
nome, colecao, numero	string	Nome da carta, coleção e número na coleção
tipo	Tipo	'Fogo' | 'Água' | 'Planta' | 'Elétrico' | 'Psíquico' | 'Fada'
raridade	Raridade	'Comum' | 'Rara' | 'Ultra Rara' | 'Secreta' | 'Promo'
idioma, variante	string	Idioma da impressão e variante/edição (Holo, 1ª Edição...)
estado	Estado	Conservação: 'M' | 'NM' | 'EX' | 'GD' | 'PL'
preco, estoque, ano	number	Valor estimado em R$, unidades disponíveis e ano de lançamento

gradiente	Gradiente	Classe CSS da arte ('g1' a 'g6'), usada no lugar de imagem
dataCadastro	string (ISO)	Data de inclusão no acervo (exibida com o pipe `date`)

O mesmo arquivo define `FiltrosCard` (raridade, tipo e coleção usados na pesquisa) e `Contagem` (par chave/quantidade usado pelo dashboard).

3.3 Mock (src/app/mocks/cards.mock.ts)

A constante `CARDS_MOCK: Card[]` contém 12 cartas fictícias (ex.: *Pyrodrake Kasumi Holo*, *Voltilo Tribute*, *Psymora Gold*) distribuídas em 8 coleções, 6 tipos e 5 raridades. O `CardService` faz uma cópia desse array ao iniciar e passa a ser o único ponto de acesso aos dados; os componentes nunca importam o mock nem manipulam o array diretamente.

3.4 CardService (src/app/services/card.service.ts — `providedIn: 'root'`)

Método	O que faz
`listar(): Card[]`	Retorna cópias de todas as cartas do acervo.
`buscarPorId(id)`	Retorna a carta com o id informado (ou `undefined`); usado no detalhe e na edição.
`pesquisar(termo, filtros)`	Busca o termo em nome, número e coleção (sem diferenciar maiúsculas) e aplica os filtros de raridade, tipo e coleção.
`listarColecoes()`	Lista as coleções distintas, ordenadas, para os selects de filtro e o formulário.
`cadastrar(card)`	Valida, gera o próximo id e a data de cadastro e adiciona a carta.
`editar(card)`	Valida e substitui a carta de mesmo id; lança erro se não existir.
`excluir(id)`	Remove a carta e retorna `true` se algo foi excluído.
`total()`	Quantidade de cartas distintas cadastradas.
`totalUnidades()`	Soma do estoque de todas as cartas.
`valorAcervo()`	Soma de preço × estoque (valor estimado do acervo).
`contagemPorRaridade()`	Quantidade de cartas por raridade (gráfico de distribuição).

`contagemPorColecao()`	Quantidade de cartas por coleção, em ordem decrescente (gráfico de barras).
`maisValiosos(limite)`	As *n* cartas de maior preço (tabela do dashboard).
`historicoPreco(card)` / `variacaoPreco(card)`	Série simulada de 10 meses terminando no preço atual e sua variação percentual (gráfico do detalhe).
`validar(card)` *(privado)*	Regras de negócio: nome e coleção obrigatórios, preço > 0 e estoque ≥ 0.

Também foi criado o **AuthService**, com usuários mock (`cliente@cardvault.jp` e `admin@cardvault.jp`) e os métodos `login(email, senha, perfil)`, `registrar()`, `logout()`, `estaLogado()` e `isAdmin()`, que servirá de base para os Route Guards.

3.5 Rotas (src/app/app-routing.module.ts)

Caminho	Componente / ação
`''`	redirectTo `/catalogo` (pathMatch: 'full')
`catalogo`	CatalogoComponent
`cards/:id`	DetalheCardComponent (lê o parâmetro com `ActivatedRoute.paramMap`)
`admin/cards`	AdminCardsComponent (aceita `?editar=id` para abrir a edição)
`login`	LoginComponent
`dashboard`	DashboardComponent
`sobre`	SobreComponent
`**`	redirectTo `/catalogo` (rota coringa)

Navegação programática (`Router.navigate`): clicar em um card do catálogo ou do top 5 do dashboard abre `['/cards', id]`; os botões Voltar e Editar do detalhe vão para o catálogo e para `/admin/cards?editar=id`; salvar no admin volta para a lista `/admin/cards`; o login redireciona conforme o perfil e o logout da navbar volta para `/login`.

3.6 Diretivas, data-binding e pipes utilizados

Recurso	Exemplo no código	Onde aparece
Interpolação `{{ }}`	`{{ card.nome }}`, `{{ totalAcervo }} cartas no acervo`	Todas as páginas e navbar
Property binding `[ ]`	`[routerLink]="link.rota"`, `[value]="r"`,	Navbar, filtros, detalhe (SVG), dashboard,

	`[disabled]="propostaEnviada"`, `[attr.points]`, `[style.width.%]`	admin
Event binding `( )`	`(click)="abrirDetalhe(card)"`, `(ngSubmit)="salvar(f)"`, `(ngModelChange)="filtrar()"`	Catálogo, detalhe, admin, login, sobre, navbar
Two-way `[(ngModel)]`	`[(ngModel)]="termo"`, `[(ngModel)]="modelo.preco"`	Busca/filtros do catálogo, formulário e filtro do admin, login, contato
`*ngFor`	`*ngFor="let card of cards"`	Grade do catálogo, tabela do admin, links da navbar, gráficos do dashboard, opções dos selects
`*ngIf` (com `else`)	`*ngIf="cards.length > 0; else vazio"`, `*ngIf="card as c; else naoEncontrado"`	Catálogo, detalhe, mensagens de validação e de sucesso, navbar (logado/deslogado)
`[ngClass]`	`[ngClass]="card.gradiente"`, `[ngClass]="{ err: nome.invalid && ... }"`	Arte das cartas, tags de raridade, estados de validação, abas do login
`[ngStyle]`	`[ngStyle]="{ height: ... + 'px' }"`, `[ngStyle]="{ opacity: m }"`	Barras do dashboard, miniaturas do detalhe, cartas da vitrine do login, estoque zerado no admin
Pipe `currency`	`{{ card.preco | currency:'BRL' }}` (locale pt-BR)	Catálogo, detalhe, admin, dashboard
Pipes `date` e `number`	`{{ c.dataCadastro | date:'dd/MM/yyyy' }}`, `{{ variacao | number:'1.1-1' }}`	Detalhe e dashboard
Pipe próprio `conservacao`	`{{ c.estado | conservacao }}` → "Near Mint (NM)"	Detalhe, select de estado do admin, dashboard
Template reference / validação	`#f="ngForm"`, `#nome="ngModel"`, `required`, `minlength`, `min="0.01"`, `email`, `pattern`	Admin, login, sobre
`routerLink` / `routerLinkActive`	`routerLinkActive="on"`	Navbar e trilha do detalhe

4. TELAS

Os prints a seguir foram capturados com a aplicação em execução (`ng serve`) na resolução 1440×900, usando Playwright, com o usuário administrador logado.

4.1 Login (/login)

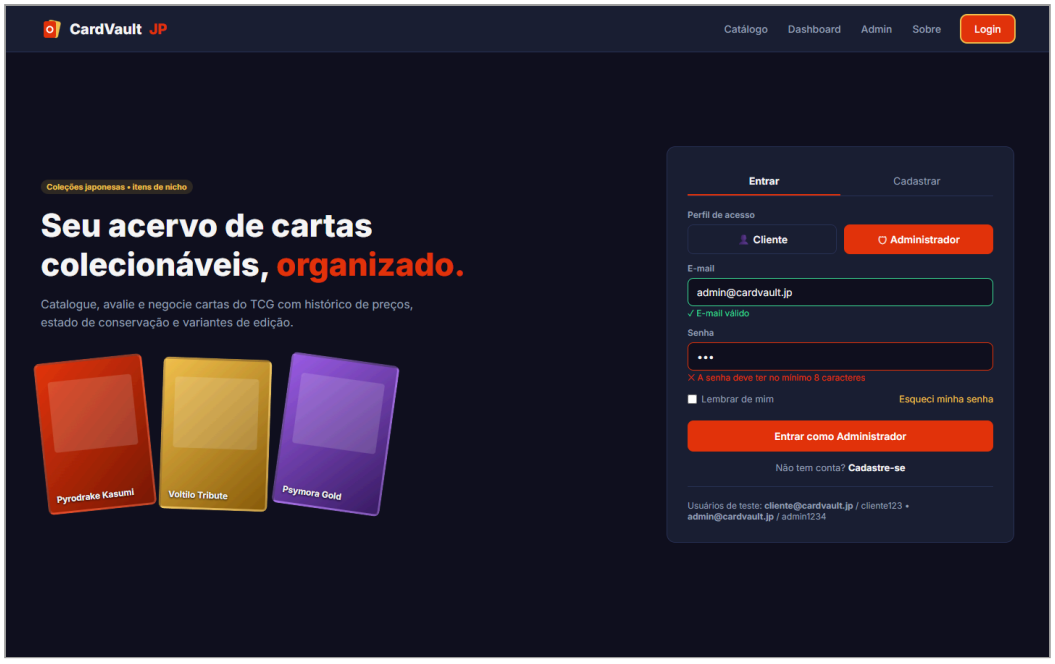


Figura 1 — Login com validação de senha (mínimo de 8 caracteres)

- Abas Entrar/Cadastrar alternadas com `[ngClass]` e `*ngIf`;
- Seleção de perfil Cliente/Administrador e autenticação via `AuthService` (usuários mock);
- Validação de e-mail (formato com domínio) e senha (mínimo 8), com feedback verde/vermelho;
- Após o login, `Router.navigate` leva o admin para `/admin/cards` e o cliente para `/catalogo`; a navbar passa a exibir o usuário e o botão Sair;
- Aba Cadastrar com nome, e-mail, senha e confirmação (verifica se as senhas conferem).

4.2 Catálogo (/catalogo)

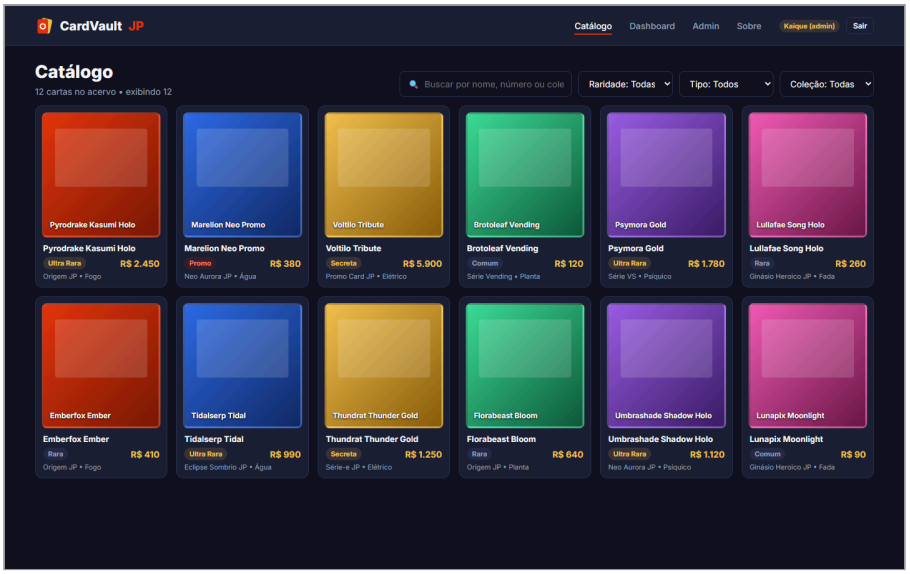


Figura 2 — Catálogo com os 12 cards do acervo

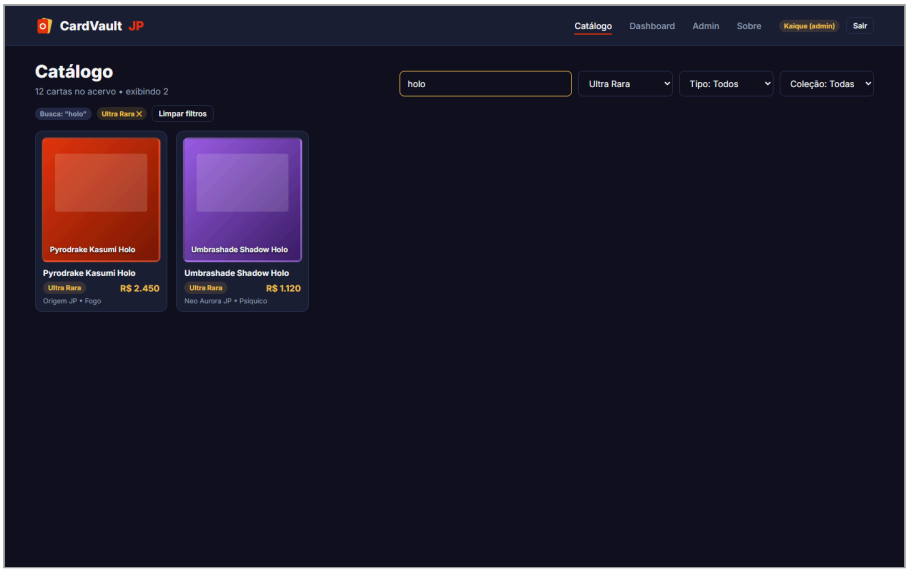


Figura 3 — Busca "holo" combinada com o filtro Ultra Rara

- Grade de cards gerada com `*ngFor` a partir de `CardService.pesquisar()`;
- Busca por nome, número ou coleção e filtros de raridade, tipo e coleção com `[(ngModel)]`, atualizando a lista a cada alteração;
- Tags dos filtros ativos (clicáveis para remover) e botão Limpar filtros; mensagem quando nenhum card é encontrado;
- Preço formatado com `currency:'BRL'` e cor da raridade com `[ngClass]`;
- Clique no card abre o detalhe com `Router.navigate(['/cards', id])`.

4.3 Detalhe do card (/cards/:id)

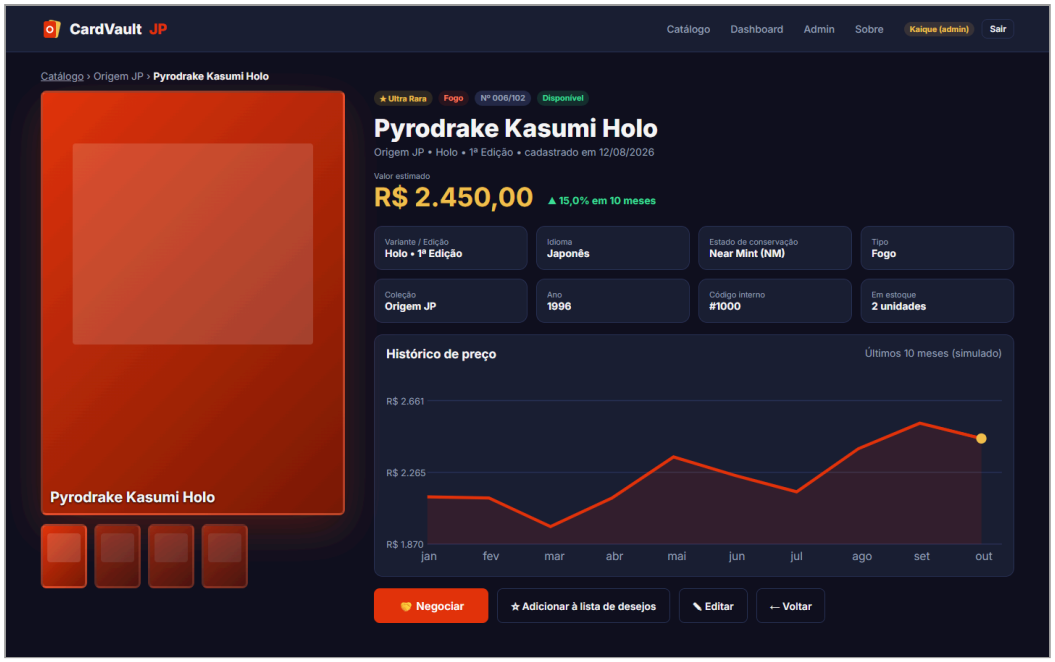


Figura 4 — Detalhe do card #1000

- Lê o `:id` com `ActivatedRoute` e obtém os dados com `CardService.buscarPorId()`; exibe mensagem se o id não existir;
- Atributos: variante, idioma, estado (pipe `conservacao`), tipo, coleção, ano, código e estoque; data de cadastro com `date`;
- Gráfico SVG do histórico de preço gerado com property binding (`[attr.points]`) e variação percentual;
- Botão Negociar (exibe confirmação da proposta e fica desabilitado), lista de desejos, Editar (abre o admin já em modo de edição) e Voltar.

4.4 Cadastro administrativo (/admin/cards)

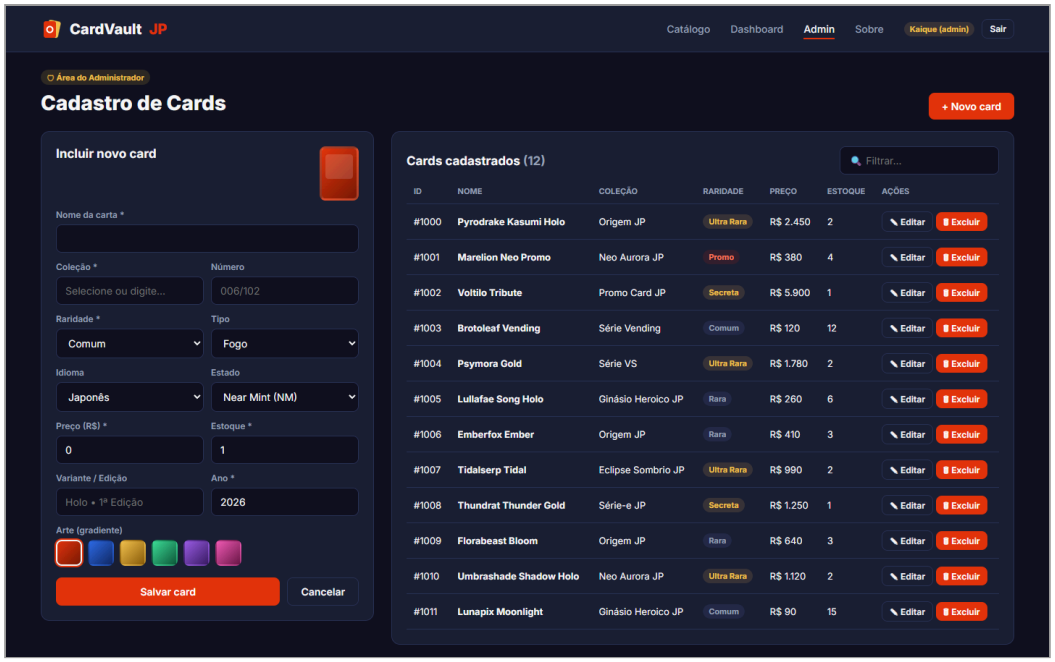


Figura 5 — Formulário de inclusão e tabela de cards cadastrados

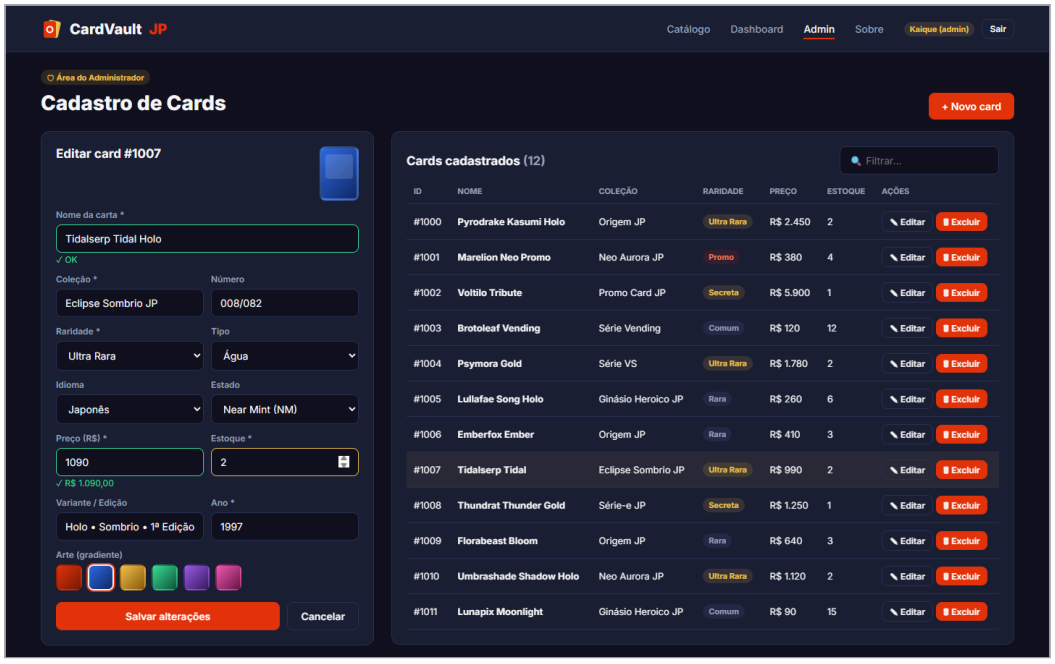


Figura 6 — Edição do card #1007 com o formulário preenchido

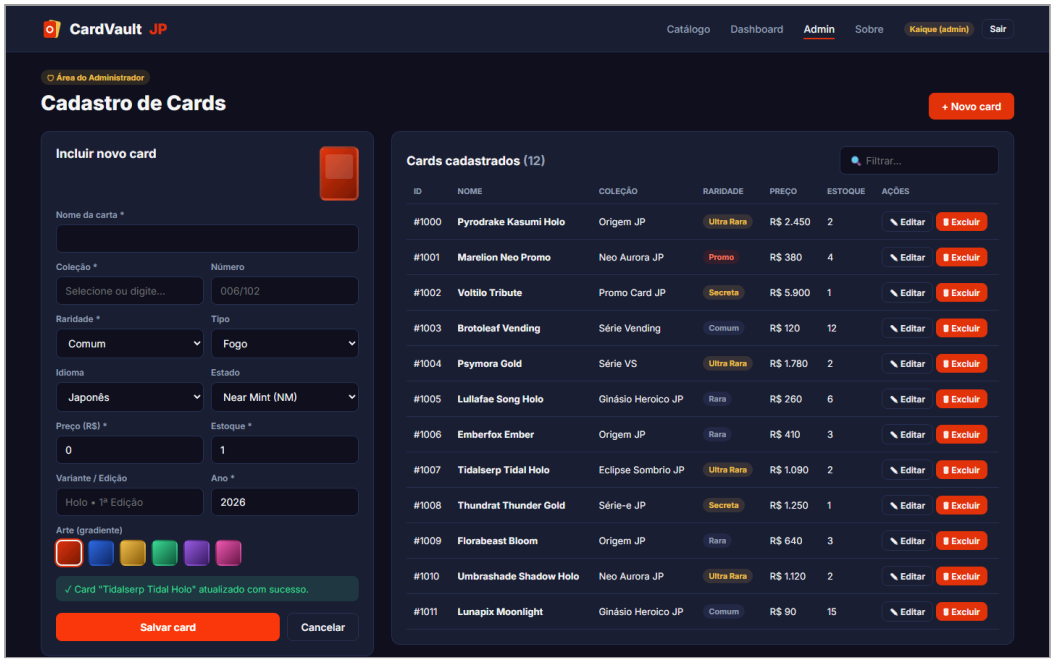


Figura 7 — Após salvar: mensagem de sucesso e tabela atualizada (nome e preço do #1007)

- Formulário template-driven (`#f="ngForm"`) para cadastrar e editar, com prévia da arte atualizada em tempo real;
- Validações: nome (obrigatório, mín. 3), coleção obrigatória, preço obrigatório e maior que zero, estoque ≥ 0 e ano entre 1996 e o ano atual; mensagens exibidas com `*ngIf` e destaque com `[ngClass]` ;
- Tabela com `*ngFor` , filtro rápido e botões Editar (carrega o card no formulário) e Excluir (com `confirm()`);
- Toda operação passa pelo `CardService` (`cadastrar` , `editar` , `excluir`); o CRUD funciona de verdade durante a sessão e reflete no catálogo e no dashboard;
- Após salvar, o formulário é limpo e `Router.navigate(['/admin/cards'])` volta para a lista.

4.5 Dashboard (/dashboard)

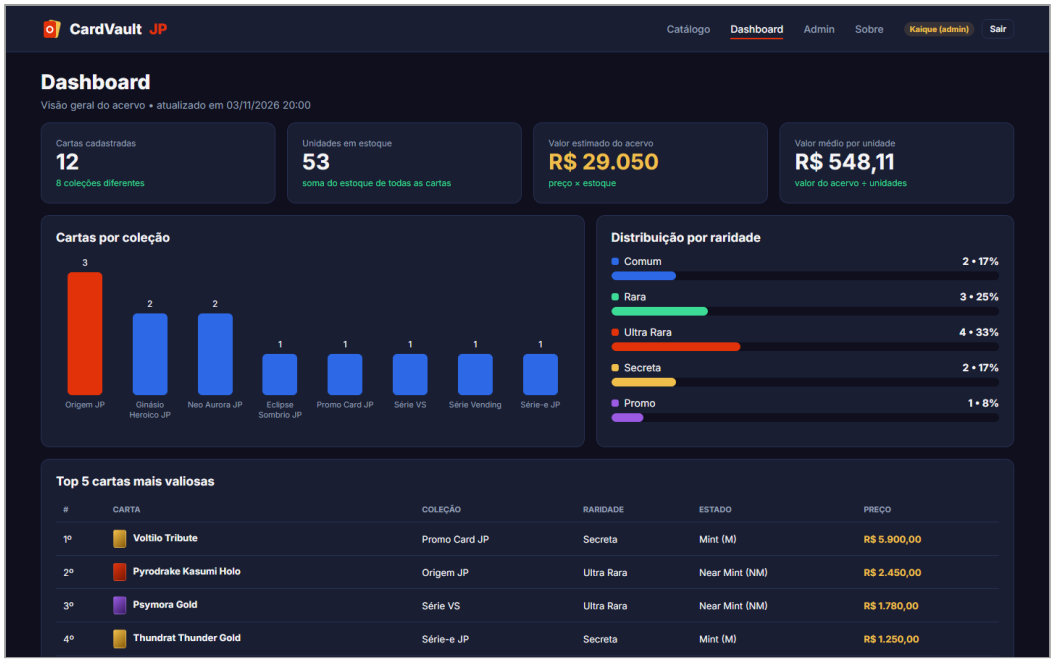


Figura 8 — Dashboard calculado pelo CardService

- KPIs calculados no serviço: cartas cadastradas, unidades em estoque, valor estimado do acervo e valor médio por unidade;
- Gráfico de barras por coleção (`[ngStyle]` para a altura) e distribuição por raridade (`[style.width.%]`);
- Top 5 cartas mais valiosas, com clique abrindo o detalhe;
- Os números mudam automaticamente após cadastros, edições e exclusões no admin.

4.6 Sobre (/sobre)

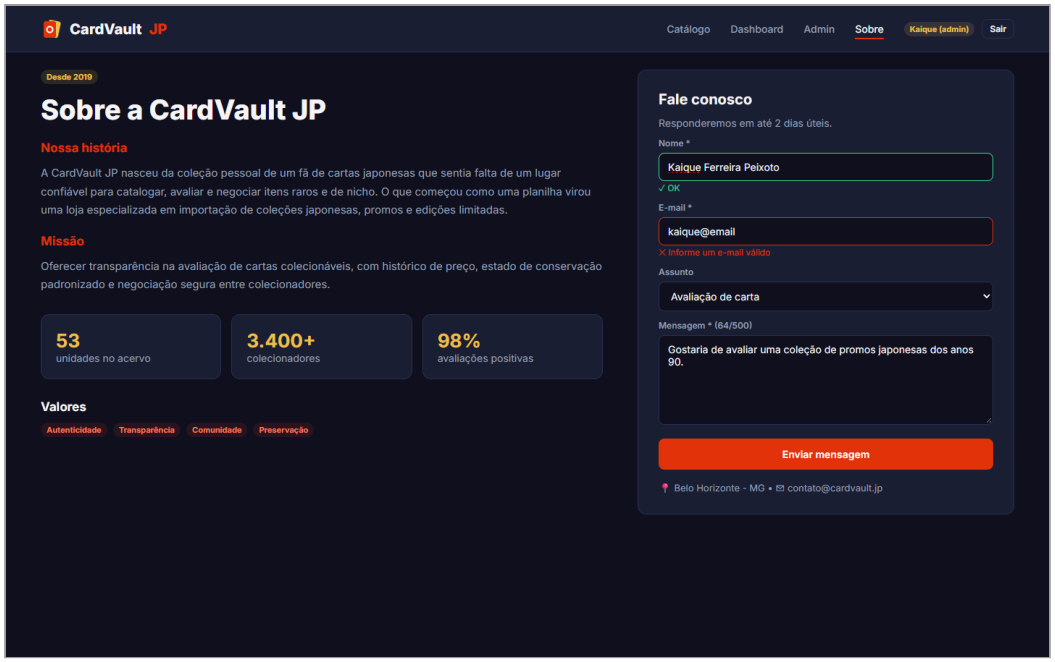


Figura 9 — Página institucional com validação do formulário de contato

- História, missão, números (unidades no acervo vindas do `CardService`) e valores gerados com `*ngFor`;
- Formulário de contato com `[(ngModel)]`, validação de nome, e-mail e mensagem (mín. 10 caracteres), contador de caracteres e mensagem de envio.

5. VERSIONAMENTO

Repositório: https://github.com/IsseiSenju/cardvault-jp

GitFlow: o trabalho foi feito na `develop`, com uma branch `feature/*` por bloco, integrada de volta com `merge --no-ff`. Ao final, a `develop` foi integrada na `main` e marcada com a tag `v0.2.0`.

Branch	Conteúdo
main	Versões estáveis entregues (tag v0.2.0)
develop	Integração das funcionalidades
feature/prototipo	Trabalho 01 — protótipo HTML/CSS
feature/setup-angular	Criação do projeto Angular (NgModule), .gitignore e identidade visual
feature/card-service	Model Card, mock, CardService e AuthService
feature/catalogo	Navbar, rotas e página de catálogo
feature/detalhe	Pipe conservacao, histórico de preço e página de detalhe
feature/admin-crud	Formulário de cadastro/edição e tabela com exclusão
feature/paginas-institucionais	Login, dashboard e sobre
feature/documentacao	README, prints e este relatório

Conventional Commits: mensagens no formato `tipo(escopo): descrição` (`feat`, `fix`, `style`, `refactor`, `docs`, `chore`). Lista real de commits (`git log --oneline`, do mais recente para o mais antigo) no momento da geração deste relatório:

Hash	Mensagem
`79ac8b6`	docs(relatorio): ajusta quebras de página e tamanho das figuras
`1bab0d8`	docs(relatorio): adiciona fonte HTML do relatório do trabalho 02
`17b735d`	docs(prints): adiciona prints 1440x900 das telas da aplicação
`fd4e998`	docs: atualiza README com instruções, rotas e arquitetura do trabalho 02
`8cc26d8`	merge: integra feature/paginas-institucionais na develop
`3f74787`	fix(forms): exige domínio com ponto na validação de e-mail
`c26bcae`	style(detalhe): ajusta gráfico de histórico para ocupar toda a largura
`a244784`	refactor(rotas): agrupa imports dos componentes no módulo de rotas
`3f299a9`	feat(sobre): adiciona página institucional com formulário de contato

`c7ae27a`	feat(dashboard): exibe indicadores e gráficos a partir do CardService
`1cbe7f6`	feat(login): adiciona login com perfis e cadastro de cliente
`649721d`	feat(auth): permite cadastro de novos clientes no AuthService
`bf0ff1a`	merge: integra feature/admin-crud na develop
`097ca54`	feat(admin): implementa cadastro e edição de cards com validação
`b4e209e`	merge: integra feature/detalhe na develop
`66cd7a0`	feat(detalhe): exibe dados do card pela rota /cards/:id
`d4b7316`	feat(pipes): cria pipe conservacao para exibir o estado da carta
`4c461c5`	feat(services): adiciona histórico de preço simulado e variação percentual
`bdb94ef`	merge: integra feature/catalogo na develop
`88f0fcf`	feat(rotas): configura rotas com redirect para catálogo e rota coringa
`70f19ec`	feat(catalogo): lista cards com filtros e busca
`d7e89ce`	feat(navbar): adiciona navbar com routerLink e routerLinkActive
`d631ced`	merge: integra feature/card-service na develop
`f094582`	feat(services): adiciona AuthService com mock de usuários cliente e admin
`e046156`	feat(services): implementa CardService com CRUD e pesquisa
`293eedb`	feat(models): adiciona interface Card e mock de cartas
`db36d6c`	merge: integra feature/setup-angular na develop
`022551b`	style: aplica identidade visual do protótipo
`971b4fa`	chore: cria projeto angular com módulo de rotas
`f08286b`	chore: adiciona .gitignore excluindo node_modules, .angular e dist
`ba1e7e2`	merge: integra feature/prototipo na develop
`32c3b67`	docs(relatorio): corrige link do repositório
`62e0aa6`	merge: integra feature/prototipo na develop
`209818d`	docs(relatorio): remove campo de link do Figma
`f05cf7a`	merge: integra feature/prototipo na develop
`15eff15`	chore(kanban): atualiza kanban do trabalho 01
`b413b26`	docs(relatorio): adiciona relatório do trabalho 01
`ac6d4c4`	feat(prototipo): adiciona tela de institucional
`161ea61`	feat(prototipo): adiciona tela de dashboard
`9f6d83b`	feat(prototipo): adiciona tela de cadastro administrativo
`cb5ea42`	chore: inicializa repositório
`65990fa`	feat(prototipo): adiciona tela de detalhes do card
`7b6dbbd`	feat(prototipo): adiciona tela de catálogo
`34fc876`	feat(prototipo): adiciona tela de login e cadastro

Após a geração deste relatório foram feitos ainda o commit que adiciona este PDF, os merges de `feature/documentacao` em `develop` e de `develop` em `main`, e a tag `v0.2.0`.

6. PRÓXIMAS ETAPAS

- **JSON-Server:** substituir o array em memória por uma API REST simulada (`db.json` com cards, usuários e negociações), usando `HttpClient` e `Observable` no `CardService` (GET, POST, PUT e DELETE), para que os dados persistam entre sessões;
- **Route Guards:** proteger `/admin/cards` e `/dashboard` com um guard (`CanActivate`) baseado no `AuthService`, permitindo acesso apenas ao perfil Administrador e redirecionando para `/login`;
- **Lazy Loading:** mover a área administrativa (admin e dashboard) para um módulo próprio (`AdminModule`) carregado sob demanda com `loadChildren`, reduzindo o bundle inicial;
- Melhorias: persistir a sessão do usuário, fluxo completo de negociação entre colecionadores e lista de desejos por usuário.